C# Programming for Indian Students — Chapter 4: Operators and User Input

Oct 31, 2026 · @Vikash

4.1 What you will learn

In Chapter 3 every value was typed into the program itself. Real programs **calculate** and **listen**: they work out a total, compare marks with a pass mark, and read what the user types. By the end of this chapter you will be able to:

- calculate with the arithmetic operators `+`, `-`, `*`, `/` and `%`, and explain why `7 / 2` gives `3`;
- update a variable with shortcuts such as `+=` and `++`, and trace prefix and postfix forms;
- compare values with relational operators and combine conditions with `&&`, `||` and `!`;
- read input from the keyboard with `Console.ReadLine` and turn it into a number;
- convert between types with casts, and predict the order in which an expression is worked out;
- build a GST Bill Calculator that reads a price and prints a bill.

How input programs are shown. When a program asks the user to type something, the output shows the typed value after the prompt, as it appears on screen. Every program in this chapter was compiled and run on the .NET 10 SDK with the sample input shown.

Remember: all programs use top-level statements, as in Chapter 3. Each one is a list of lines you place in `Program.cs` of a new console project.

Vikash Kumar

4.2 Arithmetic operators

An operator is a symbol that tells C# to do something with one or more values. The values it works on are called **operands**. In `a + b`, the `+` is the operator and `a` and `b` are the operands. A combination such as `a + b` that produces a value is called an **expression**.

Operator	Name	Example	Result when a = 17, b = 5
`+`	Addition	`a + b`	22
`-`	Subtraction	`a - b`	12

Operator	Name	Example	Result when a = 17, b = 5
`*`	Multiplication	`a * b`	85
`/`	Division	`a / b`	3
`%`	Remainder (modulus)	`a % b`	2

Program 4.1: Arithmetic operators

```
int a = 17;
int b = 5;

Console.WriteLine($"a + b = {a + b}");
Console.WriteLine($"a - b = {a - b}");
Console.WriteLine($"a * b = {a * b}");
Console.WriteLine($"a / b = {a / b}");
Console.WriteLine($"a % b = {a % b}");
```

Output

```
a + b = 22
a - b = 12
a * b = 85
a / b = 3
a % b = 2
```

How it works. Inside `{ }` of an interpolated string you can write any expression, not only a variable name. C# works out `a + b` first and prints the result. The surprise is `17 / 5 = 3`: when **both** operands are `int`, C# does **integer division** and throws away the part after the decimal point. It does not round: 3.4 becomes 3, and 3.9 also becomes 3. The `%` operator gives what is left over: 17 = 5 × 3 + 2, so `17 % 5` is 2.

Where % is useful. `n % 2 == 0` tells you whether a number is even, `n % 10` gives the last digit of a number, and `minutes % 60` gives the minutes left after taking out whole hours. You will use these in Chapters 5 and 6.

Integer division vs decimal division. If at least one operand is a `double` or `decimal`, C# does ordinary division and keeps the fraction.

Program 4.2: Integer and decimal division

Vikash Kumar

```
int totalMarks = 437;
int subjects = 5;

Console.WriteLine(7 / 2);
Console.WriteLine(7.0 / 2);
Console.WriteLine(7 / 2.0);
Console.WriteLine(7m / 2);
Console.WriteLine($"Average (int)    : {totalMarks / subjects}");
Console.WriteLine($"Average (double) : {(double)totalMarks / subjects}");
```

Output

```
3
3.5
3.5
3.5
Average (int)    : 87
Average (double) : 87.4
```

How it works. `7 / 2` uses two `int` values, so the answer is 3. Writing `7.0` or `2.0` makes one operand a `double`, and `7m` makes it a `decimal`, so the answer keeps .5. The last line uses a **cast**, `(double)totalMarks`, to turn the `int` into a `double` before dividing. You will study casts in section 4.6.

Remember: to get a correct average, percentage or CGPA from whole numbers, make one operand a `double` (or `decimal` for money) before you divide. `437 / 5` silently loses 0.4 marks.

4.3 Assignment and increment operators

You already know the assignment operator `=`. Very often a program takes a variable, changes it a little and stores it back, for example adding a deposit to a bank balance. **Compound assignment operators** write this in a shorter way.

Vikash Kumar

Operator	Example	Same as
`+=`	`balance += 200;`	`balance = balance + 200;`
`-=`	`balance -= 150;`	`balance = balance - 150;`
`*=`	`balance *= 2;`	`balance = balance * 2;`
`/=`	`balance /= 4;`	`balance = balance / 4;`
`%=`	`balance %= 100;`	`balance = balance % 100;`

Increment and decrement. Adding or subtracting 1 is so common that C# has two special operators: `++` adds 1 and `--` subtracts 1. Each has two forms:

- **Postfix** `x++` : use the current value of `x` first, then add 1.
- **Prefix** `++x` : add 1 first, then use the new value.

When `x++` or `++x` stands alone on its own line, both forms do the same thing. The difference appears only when the result is used inside a bigger expression.

Program 4.3: Compound assignment and increment

```
int balance = 500;
balance += 200;   // balance = balance + 200
Console.WriteLine($"After deposit    : {balance}");
balance -= 150;   // balance = balance - 150
Console.WriteLine($"After withdrawal : {balance}");
balance *= 2;
Console.WriteLine($"After *= 2       : {balance}");
balance /= 4;
Console.WriteLine($"After /= 4       : {balance}");
balance %= 100;
Console.WriteLine($"After %= 100     : {balance}");

int x = 5;
int y = x++;      // postfix: use 5, then x becomes 6
Console.WriteLine($"x = {x}, y = {y}");
int z = ++x;      // prefix: x becomes 7, then use 7
Console.WriteLine($"x = {x}, z = {z}");
```

Output

Vikash Kumar

```
After deposit    : 700
After withdrawal : 550
After *= 2       : 1100
After /= 4       : 275
After %= 100     : 75
x = 6, y = 5
x = 7, z = 7
```

Dry run of the increment part

Step	Statement	x	y	z
1	`int x = 5;`	5	-	-
2	`int y = x++;`	6	5 (old value of x)	-
3	`int z = ++x;`	7	5	7 (new value of x)

Remember: `+=` has the `+` **before** the `=`. Writing `total =+ 5;` compiles without any error, but it means `total = +5;` and simply stores 5. This is a logic error that the compiler cannot catch.

4.4 Relational and logical operators

A relational operator compares two values and always gives a `bool` answer: `True` or `False`. These answers are what programs use to make decisions, which you will do with `if` in Chapter 5.

Operator	Meaning	Example (marks = 72)	Result
`==`	Equal to	`marks == 72`	True
`!=`	Not equal to	`marks != 72`	False
`>`	Greater than	`marks > 90`	False
`<`	Less than	`marks < 40`	False
`>=`	Greater than or equal to	`marks >= 40`	True
`<=`	Less than or equal to	`marks <= 100`	True

Logical operators join or reverse `bool` values. Think of a college rule: *you may sit the exam if you passed the internal test **and** your attendance is at least 75%*.

Vikash Kumar

Operator	Name	True when	Example
`&&`	AND	**both** sides are true	`passed && goodAttendance`
`||`	OR	**at least one** side is true	`passed || goodAttendance`
`!`	NOT	the value is false (it flips it)	`!passed`

Truth table

A	B	`A && B`	`A || B`	`!A`
True	True	True	True	False
True	False	False	True	False
False	True	False	True	True
False	False	False	False	True

Program 4.4: Relational and logical operators

```
int marks = 72;
int attendance = 68;
char grade = 'B';

bool passed = marks >= 40;
bool goodAttendance = attendance >= 75;

Console.WriteLine($"Passed           : {passed}");
Console.WriteLine($"Good attendance  : {goodAttendance}");
Console.WriteLine($"Grade is A       : {grade == 'A'}");
Console.WriteLine($"Grade is not A   : {grade != 'A'}");
Console.WriteLine($"Can sit the exam : {passed && goodAttendance}");
Console.WriteLine($"Gets any credit  : {passed || goodAttendance}");
Console.WriteLine($"Failed           : {!passed}");
```

Output

```
Passed           : True
Good attendance  : False
Grade is A       : False
Grade is not A   : True
Can sit the exam : False
Gets any credit  : True
Failed           : False
```

Vikash Kumar

How it works. `marks >= 40` is True and `attendance >= 75` is False. With `&&` both must be true, so the student cannot sit the exam. With `||` one true side is enough. `!passed` flips True to False.

Short-circuit evaluation. C# stops as soon as it knows the answer. In `A && B`, if A is False the result must be False, so B is never checked. In `A || B`, if A is True, B is never checked. This matters later, when B could cause an error, such as dividing by a number that might be zero.

Remember: `=` stores a value, `==` compares two values. To check a range, join two comparisons: `marks >= 40 && marks <= 100`. Writing `40 <= marks <= 100` as in mathematics is an error in C#.

4.5 Reading input from the keyboard

`Console.ReadLine()` waits for the user to type a line and press Enter, then gives back what was typed as a `string`. It always returns text, even when the user types digits.

Program 4.5: Reading text from the keyboard

```
Console.Write("Enter your name: ");
string name = Console.ReadLine() ?? "";

Console.Write("Enter your city: ");
string city = Console.ReadLine() ?? "";

Console.WriteLine($"Namaste, {name} from {city}!");
```

Output (the user typed `Priya` and `Kochi`)

```
Enter your name: Priya
Enter your city: Kochi
Namaste, Priya from Kochi!
```

How it works. `Console.Write` (not `WriteLine`) prints the prompt and keeps the cursor on the same line, so the user types right after the colon. `Console.ReadLine()` then pauses the program until Enter is pressed.

What is `?? ""`? In rare cases, such as when input comes from a file that has ended, `ReadLine` has nothing to give back and returns `null`, which means *no value*. The `??` operator says: *if the left side is null, use the right side instead*. So `Console.ReadLine() ?? ""` always gives a real string, empty if nothing was read. Without it, the compiler shows a warning because the project has `Nullable` switched on (Chapter 2). You will study null properly in a later chapter; for now, write `?? ""` after every `ReadLine`.

Vikash Kumar

Turning text into numbers. You cannot calculate with the text `"25"`. You must convert it into a number first.

Method	Example	If the text is not a number
`int.Parse(text)`	`int.Parse("25")` gives 25	Program stops with FormatException
`Convert.ToInt32(text)`	`Convert.ToInt32("25")` gives 25	Program stops with FormatException; a null value gives 0
`int.TryParse(text, out int n)`	gives True and puts 25 in `n`	No crash: gives False and puts 0 in `n`

The same methods exist for other types: `double.Parse`, `decimal.Parse`, `Convert.ToDouble`, `decimal.TryParse` and so on.

Program 4.6: Adding two numbers typed by the user

```
Console.Write("Enter first number : ");
int first = int.Parse(Console.ReadLine() ?? "");

Console.Write("Enter second number: ");
int second = Convert.ToInt32(Console.ReadLine());

int sum = first + second;
Console.WriteLine($"Sum = {sum}");
```

Output (the user typed `25` and `17`)

```
Enter first number : 25
Enter second number: 17
Sum = 42
```

How it works. The first number is converted with `int.Parse` and the second with `Convert.ToInt32`, so you can see both. If you skipped the conversion and joined the two strings with `+`, you would get `"2517"` instead of 42, because `+` on text joins it.

Program 4.7: Safe conversion with TryParse

Vikash Kumar

```
Console.Write("Enter your age: ");
string text = Console.ReadLine() ?? "";

bool isNumber = int.TryParse(text, out int age);

Console.WriteLine($"Valid number : {isNumber}");
Console.WriteLine($"Age          : {age}");
```

Output (the user typed `twenty`)

```
Enter your age: twenty
Valid number : False
Age          : 0
```

How it works. `TryParse` tries the conversion and returns `True` or `False` instead of crashing. The converted number comes out through `out int age`, which declares the variable `age` in the same line. With the input `twenty` it returns False and `age` is 0. In Chapter 5 you will use `if` to print a friendly message when the input is not valid.

Remember: `ReadLine` always gives a string. Convert it before you calculate, and prefer `TryParse` whenever a user might type something unexpected.

4.6 Type conversion and operator precedence

Converting a value from one type to another is called type conversion. C# does it in two ways.

Kind	When	Example	Data lost?
Implicit (automatic)	Small type to bigger type: `int` to `long`, `int` to `double`	`double percent = marks;`	Never
Explicit (cast)	Bigger type to smaller type: `double` to `int`, `int` to `byte`	`int rupees = (int)price;`	Possibly, and you accept it

Analogy. Pouring water from a glass into a bucket is always safe (implicit). Pouring from a bucket into a glass may spill, so C# makes you say so with a cast (explicit). Without the cast, `int i = 7.5;` is a compile-time error.

Operator precedence decides which part of an expression is worked out first, just as BODMAS does in mathematics.

Vikash Kumar

Priority	Operators	Example
1 (highest)	`( )` brackets	`(2 + 3) * 4` gives 20
2	`++` `--` `!` and casts such as `(int)`	`!passed`, `(double)total`
3	`*` `/` `%`	`7 + 10 % 3` gives 8
4	`+` `-`	`2 + 3 * 4` gives 14
5	`<` `>` `<=` `>=`	`total > 40`
6	`==` `!=`	`grade == 'A'`
7	`&&`	`a && b`
8	`||`	`a || b`
9 (lowest)	`=` `+=` `-=` and the other assignments	`sum = a + b`

Operators of equal priority are worked out from **left to right**, so `20 / 4 * 2` is `5 * 2`, which is 10. Assignment is the exception: it works right to left.

Program 4.8: Casting and precedence

```
// Implicit conversion: small type to bigger type, no data lost
int marks = 85;
double percent = marks;
Console.WriteLine($"double from int : {percent}");

// Explicit conversion (cast): you accept that data may be lost
double price = 99.75;
int rupees = (int)price;
Console.WriteLine($"(int)99.75      : {rupees}");
Console.WriteLine($"Math.Round      : {Math.Round(price)}");

int big = 300;
byte small = (byte)big;
Console.WriteLine($"(byte)300       : {small}");

// Precedence: * / % before + -, left to right for equals
Console.WriteLine($"2 + 3 * 4       : {2 + 3 * 4}");
Console.WriteLine($"(2 + 3) * 4     : {(2 + 3) * 4}");
Console.WriteLine($"20 / 4 * 2      : {20 / 4 * 2}");
Console.WriteLine($"7 + 10 % 3      : {7 + 10 % 3}");
```

Output

Vikash Kumar

```
double from int : 85
(int)99.75      : 99
Math.Round      : 100
(byte)300       : 44
2 + 3 * 4       : 14
(2 + 3) * 4     : 20
20 / 4 * 2      : 10
7 + 10 % 3      : 8
```

How it works. `(int)99.75` cuts off the fraction and gives 99; it does not round. To round, use `Math.Round`, which gives 100. `(byte)300` gives 44 because a `byte` holds only 0 to 255: the value wraps around, and 300 − 256 = 44. This is why a cast is dangerous when the value may not fit.

Remember: when in doubt, add brackets. `(a + b) / 2` and `a + b / 2` are very different, and brackets make your intention clear to the next reader.

4.7 Dry run and common errors

Dry run: operators one step at a time. Follow this program and write down the value of each variable after every line.

Program 4.9: Dry run

```
int a = 10;
int b = 3;
a += b * 2;
b = a % 5;
int c = a / b++;
bool ok = c > 15 && b == 2;
Console.WriteLine($"a = {a}, b = {b}, c = {c}, ok = {ok}");
```

Step	Statement	Working	a	b	c	ok
1	`int a = 10;`		10	-	-	-
2	`int b = 3;`		10	3	-	-
3	`a += b * 2;`	`*` first: 3 × 2 = 6, then a = 10 + 6	16	3	-	-
4	`b = a % 5;`	16 = 5 × 3 + 1, remainder 1	16	1	-	-
5	`int c = a / b++;`	postfix: uses b = 1, so c = 16 / 1; then b becomes 2	16	2	16	-
6	`bool ok = c > 15 && b == 2;`	True && True	16	2	16	True

Vikash Kumar

Output

```
a = 16, b = 2, c = 16, ok = True
```

Common errors. Each message below is the real message from the .NET 10 compiler or runtime.

Mistake	Wrong code	Message	Fix
Storing input directly in an int	`int x = Console.ReadLine();`	`error CS0029: Cannot implicitly convert type 'string' to 'int'`	`int x = int.Parse(Console.ReadLine() ?? "");`
double into int without a cast	`double d = 7.5; int i = d;`	`error CS0266: Cannot implicitly convert type 'double' to 'int'. An explicit conversion exists (are you missing a cast?)`	`int i = (int)d;` if you accept losing .5
Range written as in maths	`bool r = 40 <= marks <= 100;`	`error CS0019: Operator '<=' cannot be applied to operands of type 'bool' and 'int'`	`marks >= 40 && marks <= 100`
`=` instead of `==`	`bool r = (marks = 40);`	`error CS0029: Cannot implicitly convert type 'int' to 'bool'`	`bool r = (marks == 40);`
Dividing by a constant zero	`Console.WriteLine(10 / 0);`	`error CS0020: Division by constant zero`	Divide by a non-zero value
No `?? ""` after ReadLine	`int.Parse(Console.ReadLine())`	`warning CS8604: Possible null reference argument for parameter 's' in 'int int.Parse(string s)'.`	`int.Parse(Console.ReadLine() ?? "")`

Vikash Kumar

Run-time errors. Some mistakes compile but stop the program while it runs. The runtime prints *Unhandled exception* followed by the name of the problem.

Situation	Message when the program runs
`int b = 0;` then `a / b`	`System.DivideByZeroException: Attempted to divide by zero.`
User types `abc` for `int.Parse`	`System.FormatException: The input string 'abc' was not in a correct format.`
User types `3000000000` for `int.Parse`	`System.OverflowException: Value was either too large or too small for an Int32.`

Logic errors are the hardest: no message at all, just a wrong answer. Examples from this chapter: `total =+ 5;` (stores 5 instead of adding), `437 / 5` (gives 87 instead of 87.4) and `a + b / 2` when you meant `(a + b) / 2`.

Remember: compile-time errors stop the build, run-time errors stop the program, and logic errors stop nothing. Only a dry run or testing with known answers finds the last kind. Handling run-time errors safely is the topic of Chapter 11.

4.8 Mini project: GST Bill Calculator

Problem. A small shop wants a program that prints a bill with GST. The shopkeeper types the item name, the price of one unit, the quantity and the GST rate. The program calculates the subtotal, splits the GST equally into CGST and SGST (as on an intra-state bill), and prints the total.

Formulas

Value	Formula	For 6 notebooks at Rs. 45.50, GST 12%
Subtotal	price × quantity	45.50 × 6 = 273.00
GST amount	subtotal × rate / 100	273.00 × 12 / 100 = 32.76
CGST and SGST	GST amount / 2 each	16.38 each
Total	subtotal + GST amount	273.00 + 32.76 = 305.76

Program 4.10: GST Bill Calculator

Vikash Kumar

```
const string ShopName = "Sharma General Store";

Console.Write("Item name        : ");
string item = Console.ReadLine() ?? "";

Console.Write("Price per unit   : ");
decimal price = decimal.Parse(Console.ReadLine() ?? "");

Console.Write("Quantity         : ");
int quantity = int.Parse(Console.ReadLine() ?? "");

Console.Write("GST rate (%)     : ");
decimal gstRate = decimal.Parse(Console.ReadLine() ?? "");

decimal subtotal = price * quantity;
decimal gstAmount = subtotal * gstRate / 100;
decimal cgst = gstAmount / 2;
decimal sgst = gstAmount / 2;
decimal total = subtotal + gstAmount;

Console.WriteLine();
Console.WriteLine("==================================");
Console.WriteLine($"   {ShopName}");
Console.WriteLine("==================================");
Console.WriteLine($"Item       : {item}");
Console.WriteLine($"Rate x Qty : Rs. {price:F2} x {quantity}");
Console.WriteLine($"Subtotal   : Rs. {subtotal:F2}");
Console.WriteLine($"CGST       : Rs. {cgst:F2} ({gstRate / 2}%)");
Console.WriteLine($"SGST       : Rs. {sgst:F2} ({gstRate / 2}%)");
Console.WriteLine("----------------------------------");
Console.WriteLine($"Total      : Rs. {total:F2}");
Console.WriteLine("==================================");
```

Output (the user typed `Notebook`, `45.50`, `6` and `12`)

Vikash Kumar

```
Item name        : Notebook
Price per unit   : 45.50
Quantity         : 6
GST rate (%)     : 12

==================================
   Sharma General Store
==================================
Item       : Notebook
Rate x Qty : Rs. 45.50 x 6
Subtotal   : Rs. 273.00
CGST       : Rs. 16.38 (6%)
SGST       : Rs. 16.38 (6%)
----------------------------------
Total      : Rs. 305.76
==================================
```

How it works. Every amount uses `decimal`, because money must be exact (Chapter 3). The quantity is an `int`, and `price * quantity` multiplies a `decimal` by an `int`: C# converts the `int` to `decimal` implicitly. `gstRate / 2` prints 6 for a 12% rate. `{subtotal:F2}` always shows two digits after the point, as on a real bill. The empty `Console.WriteLine()` leaves a blank line between the questions and the bill.

Why these types?

Detail	Type	Reason
Item name	`string`	Text
Price, GST rate, amounts	`decimal`	Money and rates must be exact
Quantity	`int`	Whole units
Shop name	`const string`	Never changes while the program runs

Try it yourself

1. Run the program with GST 18% for a mobile charger costing Rs. 499. Check the total with a calculator.
2. Add a line that asks for a discount percentage and subtract the discount from the subtotal before GST.
3. Print the total rounded to the nearest rupee with `Math.Round(total)`.
4. Type `abc` for the quantity and read the run-time error. Chapter 5 shows how to stop this with `TryParse` and `if`.

Vikash Kumar

4.9 Practice questions

Try each question on your own first. The answers follow the list.

Section A: Short answer

1. What is the difference between `/` and `%`? Give the value of `23 / 4` and `23 % 4`.
2. Why does `7 / 2` give 3 in C#? How can you get 3.5?
3. What is the difference between `x++` and `++x`?
4. Explain `&&` and `||` with a college example.
5. What is the difference between `int.Parse` and `int.TryParse`?

Section B: Predict the output

6. What does this program print?

```
int a = 10;
int b = 3;
Console.WriteLine(a / b);
Console.WriteLine(a % b);
Console.WriteLine((double)a / b);
```

7. What does this program print?

```
int n = 4;
int m = n++ + 2;
Console.WriteLine($"{n} {m}");
int k = ++n * 2;
Console.WriteLine($"{n} {k}");
```

8. What does this program print?

```
int age = 19;
bool hasLicence = false;
Console.WriteLine(age >= 18 && hasLicence);
Console.WriteLine(age >= 18 || hasLicence);
Console.WriteLine(!(age < 18));
Console.WriteLine(5 + 2 * 3 - 4 / 2);
```

Section C: Find the error

9. `int marks = Console.ReadLine();`
10. `bool isTeen = 13 <= age <= 19;`

Section D: Write the program

Vikash Kumar

11. Read the principal, rate of interest and time in years, and print the simple interest and the total amount. (SI = P × R × T / 100)
12. Read a temperature in Celsius and print it in Fahrenheit. (F = C × 9 / 5 + 32)
13. Read marks in three subjects and print the total, the average with two decimals, and whether the student passed (average 40 or more).

Answers

1. `/` gives the quotient and `%` gives the remainder. `23 / 4` is 5 and `23 % 4` is 3, because 23 = 4 × 5 + 3.
2. Both operands are `int`, so C# does integer division and drops the fraction. Make one operand a `double`: `7.0 / 2` or `(double)7 / 2` gives 3.5.
3. `x++` uses the old value and then adds 1; `++x` adds 1 first and then uses the new value. They differ only when the result is used in a bigger expression.
4. `passed && goodAttendance` is true only when both are true: the student may sit the exam. `passed || hasSportsQuota` is true when at least one is true: the student gets admission.
5. `int.Parse` stops the program with a FormatException if the text is not a number. `int.TryParse` returns False instead and puts 0 in the `out` variable, so the program keeps running.
6. Output:

```
3
1
3.3333333333333335
```

The last digit 5 appears because a `double` stores an approximation of 10/3.

7. Output:

```
5 6
6 12
```

`n++ + 2` uses 4 (so m = 6), then n becomes 5. `++n * 2` makes n 6 first, so k = 12.

8. Output:

```
False
True
True
9
```

For the last line: `2 * 3` = 6 and `4 / 2` = 2 first, then 5 + 6 − 2 = 9.

Vikash Kumar

9. `error CS0029: Cannot implicitly convert type 'string' to 'int'`. Convert the text: `int marks = int.Parse(Console.ReadLine() ?? "");`
10. `error CS0019: Operator '<=' cannot be applied to operands of type 'bool' and 'int'`. Write `bool isTeen = age >= 13 && age <= 19;`
11. One possible answer:

```
Console.Write("Principal (Rs.)   : ");
decimal principal = decimal.Parse(Console.ReadLine() ?? "");

Console.Write("Rate (% per year) : ");
decimal rate = decimal.Parse(Console.ReadLine() ?? "");

Console.Write("Time (years)      : ");
int years = int.Parse(Console.ReadLine() ?? "");

decimal interest = principal * rate * years / 100;
Console.WriteLine($"Simple interest   : Rs. {interest:F2}");
Console.WriteLine($"Amount            : Rs. {principal + interest:F2}");
```

```
Principal (Rs.)   : 50000
Rate (% per year) : 7.5
Time (years)      : 3
Simple interest   : Rs. 11250.00
Amount            : Rs. 61250.00
```

12. One possible answer:

```
Console.Write("Temperature in Celsius: ");
double celsius = double.Parse(Console.ReadLine() ?? "");

double fahrenheit = celsius * 9 / 5 + 32;
Console.WriteLine($"{celsius} C = {fahrenheit} F");
```

```
Temperature in Celsius: 37
37 C = 98.6 F
```

13. One possible answer:

Vikash Kumar

```
Console.Write("Marks in subject 1: ");
int m1 = int.Parse(Console.ReadLine() ?? "");
Console.Write("Marks in subject 2: ");
int m2 = int.Parse(Console.ReadLine() ?? "");
Console.Write("Marks in subject 3: ");
int m3 = int.Parse(Console.ReadLine() ?? "");

int total = m1 + m2 + m3;
double average = total / 3.0;

Console.WriteLine($"Total   : {total}");
Console.WriteLine($"Average : {average:F2}");
Console.WriteLine($"Passed  : {average >= 40}");
```

```
Marks in subject 1: 78
Marks in subject 2: 64
Marks in subject 3: 91
Total   : 233
Average : 77.67
Passed  : True
```

Note `3.0` in `total / 3.0`: with `3` the average would lose its fraction.

4.10 MCQs and viva questions

Multiple-choice questions (answers are listed after the questions)

1. What is the value of `17 % 5`?
   - (a) 3
   - (b) 2
   - (c) 3.4
   - (d) 0
2. What is the value of `9 / 2` in C#?
   - (a) 4.5
   - (b) 5
   - (c) 4
   - (d) 4.0
3. Which operator checks whether two values are equal?
   - (a) `=`
   - (b) `==`
   - (c) `!=`

Vikash Kumar

   - (d) `=>`
4. `true && false` gives:
   - (a) True
   - (b) False
   - (c) 1
   - (d) an error
5. If `x = 5`, what is stored in `y` after `int y = x++;`?
   - (a) 5
   - (b) 6
   - (c) 4
   - (d) 0
6. `Console.ReadLine()` returns a value of type:
   - (a) int
   - (b) char
   - (c) string
   - (d) bool
7. Which method converts text to a number without crashing on bad input?
   - (a) `int.Parse`
   - (b) `Convert.ToInt32`
   - (c) `int.TryParse`
   - (d) `Console.ReadLine`
8. The value of `2 + 3 * 4` is:
   - (a) 20
   - (b) 14
   - (c) 24
   - (d) 9
9. `(int)9.99` gives:
   - (a) 10
   - (b) 9.99
   - (c) 9
   - (d) an error
10. `balance -= 100;` is the same as:
    - (a) `balance = 100 - balance;`
    - (b) `balance = -100;`

Vikash Kumar

- (c) `balance = balance - 100;`
- (d) `balance == balance - 100;`

MCQ answers: 1-b, 2-c, 3-b, 4-b, 5-a, 6-c, 7-c, 8-b, 9-c, 10-c.

Viva questions (short oral questions with model answers)

1. *What is an operator and an operand?* An operator is a symbol such as `+` that performs an action; operands are the values it works on.
2. *What is integer division?* When both operands are integers, `/` drops the fraction: `7 / 2` is 3.
3. *What is the use of the % operator?* It gives the remainder, for example to check even numbers (`n % 2 == 0`) or find the last digit (`n % 10`).
4. *What is short-circuit evaluation?* `&&` and `||` stop as soon as the answer is known, so the right side may never be checked.
5. *Why do we convert the result of ReadLine?* Because `ReadLine` always returns a string, and you cannot calculate with text.
6. *Implicit vs explicit conversion?* Implicit happens automatically when no data can be lost (`int` to `double`). Explicit needs a cast because data may be lost (`(int)99.75` gives 99).
7. *What happens if the user types letters for int.Parse?* The program stops with a FormatException at run time.

4.11 Summary and what comes next

- Arithmetic operators `+ - * / %` calculate. With two `int` values, `/` drops the fraction and `%` gives the remainder.
- Make one operand a `double` or `decimal` (or use a cast) to keep the fraction, for example in averages and percentages.
- `+=`, `-=` and the other compound operators update a variable. `x++` uses the old value then adds 1; `++x` adds 1 first.
- Relational operators (`== != < > <= >=`) give `bool` answers; `&&`, `||` and `!` combine or reverse them.
- `Console.ReadLine()` always returns a string. Convert it with `int.Parse`, `Convert.ToInt32` or, safely, `int.TryParse`, and write `?? ""` after `ReadLine`.
- Implicit conversion is automatic and safe; a cast such as `(int)price` is explicit and may lose data.
- Precedence: brackets, then `* / %`, then `+ -`, then comparisons, then `&&`, then `||`, then assignment. When in doubt, add brackets.

Vikash Kumar

Next chapter: Chapter 5, **Decision Making**, uses these `bool` answers to choose what a program does. You will write `if`, `else if`, `else` and `switch`, print a friendly message when input is not valid, and build a result calculator that assigns grades from marks.

Vikash Kumar